

Жасанды интеллект бастамалары №3 Практикалық жұмыс



Тақырыбы: Жергілікті іздеу: Hill Climbing, Random Restart және Simulated Annealing

Мақсаты:

Бұл практикалық жұмыстың мақсаты – жергілікті іздеудің ағымдағы шешімді қалай жақсартатынын түсіну және дәрісте қарастырылған негізгі алгоритмдерді жүзеге асыру.

- жолды іздеу мен жергілікті іздеудің айырмашылығын ажырату;
- күйді, көршілес күйлерді және мақсаттық/мән функциясын анықтау;
- Hill Climbing алгоритмінің орындалуын қадам бойынша бақылау және оны жүзеге асыру;
- жергілікті және глобал максимумдарды түсіндіру ;
- Random Restart әдісін қолдану;
- Simulated Annealing әдісінің негізгі идеясын түсіндіру.

Hill Climbing

Ағымдағы индекстің сол және оң көршісін қарайды. Тек мәні үлкен көршіге өтеді. Үлкен көрші болмаса, тоқтайды.

Бір бастапқы шешімді таңдаймыз.

Оған ұқсас көрші шешімдерді құрамыз.

Бағасы жақсы көршіге көшеміз.

Жақсару болмаса, тоқтаймыз.

Simulated Annealing

Кездейсоқ көрші таңдалады. Жақсы қадам әрқашан қабылданады. Нашар қадам бастапқы температура жоғары кезде ықтималдықпен қабылданады.

Бірнеше бастапқы шешім таңдаймыз.

Әрқайсысынан Hill Climbing іске қосамыз.

Табылған нәтижелердің ең жақсысын аламыз.

Random Restart

Кездейсоқ 5 бастапқы индекстен Hill Climbing іске қосылады. Ең үлкен алынған нәтиже таңдалады.

Көрші шешімдерді Hill Climbing сияқты тексереді.

Кейде бағасы төмен шешімді де қабылдайды.

Қабылдау ықтималдығын температура T басқарады.

Уақыт өткен сайын T азаяды

Hill Climbing

```
def hill_climbing(values, start):  
    current = start  
    while True:  
        neighbors =  
get_neighbors(current)  
        best = choose_best(neighbors)  
        if value(best) <=  
value(current):  
            return current  
        current = best
```

Random Restart

```
import random
```

```
for _ in range(attempts):  
    start = random.randint(0,  
len(values)-1)  
    position = hill_climbing(values,  
start)  
    keep_best(position)
```

Simulated Annealing

```
import random, math
def anneal(values, start, T=10):
    current = start
    while T > 0.1:
        next_state =
random_neighbor(current)
        dE = values [next_state] - values
[current]
        if dE > 0 or random.random() <
math.exp(dE/T)
            current = next state
        T *= 0.95
    return current
```

1-тапсырма: Hill Climbing алгоритмін қолмен қадамдап орындау

values = [1, 3, 5, 8, 6, 4, 2], бастау индексі = 4

Қадам	Ағымдағы индекс	Ағымдағы мән	Көршілер	Таңдалған келесі күй
1	4	6	i=3: 8, i=5: 4	i=3, мәні 8
2	3	8	i=2: 5, i=4: 6	Тоқтау, үлкен көрші жоқ

Қорытынды

4-индекстегі 6 мәнінен 3-индекстегі 8 мәніне өтеміз. 8-дің екі көршісі де кіші болғандықтан, алгоритм осы жерде тоқтайды. **Соңғы индекс: 3 , соңғы мән: 8**

2-тапсырма: Hill Climbing кодын іске қосу

values = [1, 3, 5, 8, 6, 4, 2]

Бастапқы индекс	Өту жолы	Соңғы индекс	Соңғы мән
0	$0 \rightarrow 1 \rightarrow 2 \rightarrow 3$	3	8
2	$2 \rightarrow 3$	3	8
4	$4 \rightarrow 3$	3	8
6	$6 \rightarrow 5 \rightarrow 4 \rightarrow 3$	3	8

https://colab.research.google.com/drive/1K87zXjDd7E5mh3IRizmhVA_3xa3K5wXN?usp=sharing

Қорытынды

Бұл массивте бастапқы күй соңғы нәтижеге әсер етпейді: барлық бастау нүктесі 3-индекстегі 8 мәніне жетеді. Себебі мәндер сол жақтан да, оң жақтан да осы бір шыңға қарай өседі.

3-тапсырма: Жергілікті максимумды көрсету

values = [1, 4, 7, 5, 3, 6, 9, 8]

Бастау	Өту жолы	Соңғы индекс	Соңғы мән	Түсініктеме
0	0 → 1 → 2	2	7	Жергілікті максимум
4	4 → 5 → 6	6	9	Глобал максимум

Қорытынды

start = 0 іске қоссақ 2-индексте тоқтайды, өйткені оның екі көршісі 4 және 5. Екеуі де 7-ден кіші. Алгоритм төмен қарай уақытша қадам жасамайтындықтан, 6-индекстегі 9-ға өте алмайды. Ал start = 4 іске қоссақ 6 индексте тоқтайды және барлық күйдің максимум жақсы күйі болады.

Жергілікті максимум: индекс 2, мәні 7 Глобал максимум: индекс 6, мәні 9

4-тапсырма: Random Restart

values = [1, 4, 7, 5, 3, 6, 9, 8], attempts = 5

Эксперимент	5 бастапқы индекс	Әр әрекеттің соңғы мәні	Ең жақсы мән
1	1, 2, 4, 5, 0	7, 7, 9, 9, 7	9
2	1, 3, 1, 5, 5	7, 7, 7, 9, 9	9
3	1, 4, 6, 6, 3	7, 9, 9, 9, 7	9

<https://colab.research.google.com/drive/1QFxGFbr8f5bCAIZKn8bkXJrt276zUaaF?usp=sharing>

Қорытынды

Random Restart әр жолы бірнеше аймақтан бастайды. Бір бастау 7 жергілікті максимумына түскенде, басқа бастау 9 глобал максимумына жетуі мүмкін. Сондықтан бір бастапқы күйдің сәтсіздігі жалпы нәтижені шешпейді.

5-тапсырма: Simulated Annealing

values = [1, 4, 7, 5, 3, 6, 9, 8], бастау индексі = 0

Іске қосу	Соңғы индекс	Соңғы мән
1	2	7
2	6	9
3	6	9
4	6	9
5	2	7

https://colab.research.google.com/drive/1Wjsby0RW3vuTiy7e4PqbkFeU1n_3JhhJ?usp=sharing

Қорытынды

Барлығының соңғы мәні бірдей емес, Нәтижелер: 7, 9, 9, 9, 7. Әр қадамда көрші кездейсоқ таңдалады. Бұдан бөлек, температура жоғары кезде алгоритм кейбір нашар қадамды қабылдай алады. Осы екі кездейсоқ шешім іздеу жолын өзгертіп, әртүрлі соңғы мән беруі мүмкін.

6-тапсырма: әдістерді қысқаша салыстыру

Әдіс	Бастапқы күйлер / іске қосулар саны	Нашар қадамды қабылдау	Негізгі артықшылығы	Негізгі шектеуі
Hill Climbing	1 бастау, 1 іске қосу	Жоқ	Жылдам және қарапайым	Жергілікті максимумда тоқтайды
Random Restart	Бірнеше бастау, бұл жерде 5	Жоқ	Бірнеше аймақты тексереді	Көбірек есептеу қажет
Simulated Annealing	1 бастау, бірнеше итерация	Иә, ықтималдықпен	Жергілікті максимумнан шығуы мүмкін	Параметр мен кездейсоқтыққа сәйкес болады

Қорытынды

Hill Climbing ең жеңіл әдіс, бірақ жергілікті максимумға тәуелді. Random Restart бірнеше бастау арқылы бұл тәуелділікті азайтады. Simulated Annealing кейде төмен мәнге өтіп, жақсырақ аймақты іздей алады.

- *Жергілікті іздеу дайын шешімді біртіндеп жақсартады.
- *Hill Climbing әр қадамда жақсырақ көрші шешімге көшеді.
- *Random Restart бірнеше бастапқы шешімді тексереді.
- *Simulated Annealing кейде нашар қадамды қабылдап, тұзақтан шығады.

Қай жергілікті іздеу әдісін таңдау керек?

ҚАРАПАЙЫМ: Hill Climbing - Көршілерді бағалау оңай болса.

БІРНЕШЕ БАСТАУ: Random Restart - Әртүрлі бастапқы шешімдерді тексеру қажет болса.

ТҰЗАҚТАН ШЫҒУ: Simulated Annealing - Кейде нашар қадамға өту қажет болса.